Programação III (PG3)

Semestre de Inverno de 2026-2027

1º Trabalho prático

Data de Entrega: 30 de outubro de 2027

OBJETIVOS: Implementar aplicações simples usando o paradigma da Programação Orientada por Objetos.

NOTA: a entrega do trabalho tem de incluir todo o código desenvolvido, incluindo os testes unitários que permitem validar a correção dos métodos e classes realizadas.

O trabalho está organizado em 3 grupos que devem ser implementados sequencialmente. Os grupos são independentes, entre si. É fornecido em anexo a este enunciado testes unitários que não podem ser alterados.

Grupo 1

1. Considere a definição da classe `Rect` que representa um retângulo, caracterizado por uma largura (`w`) e uma altura (`h`):

- Indique o resultado da execução do método `main`, justificando o resultado de cada um dos `System.out`.
- Adicione o que for necessário à classe `Rect` para que o resultado da execução do método `main` seja o seguinte:

```
> java i2627.trab1.grupo1.Rect
3 x 4
3 x 4
false
true
true
```

```java
package i2627.trab1.grupo1;
public class Rect {
  private final double w;
  private final double h;
  public Rect(double w, double h) {
    this.w = w; this.h = h;
  }
  public static void main(String[] args){
    Rect r1 = new Rect(3, 4);
    Rect r2 = new Rect(3, 4);
    Rect r3 = r1;
    System.out.println( r1.toString() );
    System.out.println( r1 );
    System.out.println( r1 == r2 );
    System.out.println( r1 == r3 );
    System.out.println( r1.equals(r2) );
  }
}
```

- Justifique os dois primeiros `System.out` do método `main` produzirem o mesmo resultado no *standard output*.

2. Adicione ou altere os membros na classe `Rect`, considerando os seguintes requisitos:

- As coordenadas de origem `x` e `y`, igualmente do tipo `double`, representam a posição do canto superior esquerdo num plano;
- O construtor com 4 parâmetros, com a ordem `x`, `y`, `w`, e `h`, estabelece a origem, a largura e a altura;
- O construtor com dois parâmetros (`w` e `h`) considera a origem do retângulo no ponto 0,0. A implementação deste construtor deve chamar o construtor com quatro parâmetros;
- As coordenadas `x` e `y` podem corresponder a valores negativos mas considere que `w` e `h` são sempre positivos.
- O número de instâncias criadas com o construtor de dois parâmetros é contabilizado e o método `getNoOriginCounter` retorna o valor desse contador;
- Os métodos `getX`, `getY`, `getW` e `getH` são métodos *getters/accessors*;
- O método `toString` retorna uma `String` com o formato: `(x1,y1)->(x2,y2)`. Os pares `x1`, `y1` e `x2`, `y2` identificam as coordenadas do canto superior esquerdo e do canto inferior direito, respetivamente. Os valores devem ser apresentados com exatamente uma casa decimal;
- O método de instância `equals` retorna `true` se os dois retângulos tiverem a mesma origem, largura e altura. Considere dois retângulos iguais se a diferença entre respetivas componentes não for superior a 0.01;
- O método de instância `isOverlapped` recebe como parâmetro a referência para outro retângulo e retorna `true` se os dois retângulos estiverem totalmente ou parcialmente sobrepostos;
- O método estático `parseRect` recebe a descrição de um retângulo como parâmetro, do tipo `String`, e retorna uma instância de `Rect`. O formato da descrição é igual ao formato produzido pelo método `toString`. No caso de formatação inválida, o método retorna `null`.

  Nota: sugere-se a utilização dos métodos de instância da classe `java.lang.String`:

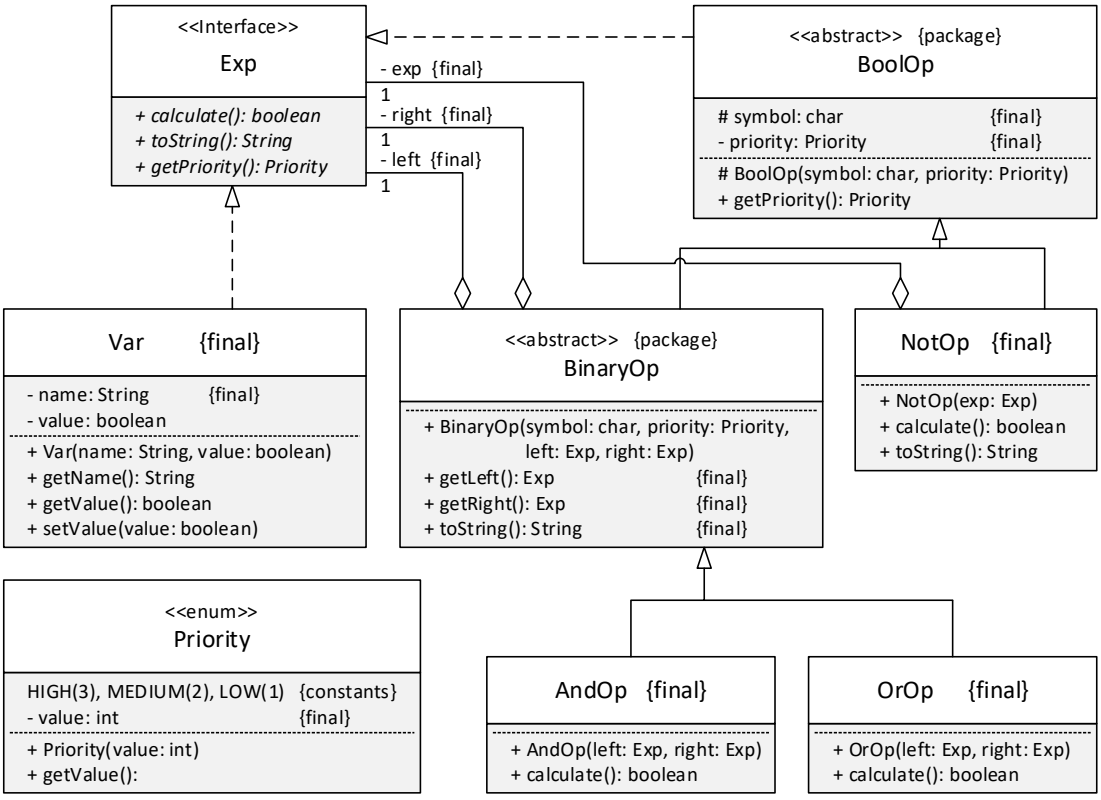
  `String[] split( String regex )` – para individualizar partes de uma *string*;

  `String substring( int beginIndex, int endIndex )` – para extrair uma *substring*;

  `String substring( int beginIndex )` – para extrair uma *substring*.

Grupo 2

Pretende-se implementar um sistema que avalia expressões booleanas baseadas nas operações *AND*, *OR* e *NOT*. Para tal, considere o diagrama de classes em baixo. As classes `AndOp`, `OrOp`, e `NotOp` modelam as operações booleanas e a classe `Var` modela uma variável da expressão. Qualquer uma destas operações, incluindo uma variável, têm associada uma prioridade, sendo que variáveis e operações *Not* têm a prioridade mais alta (`HIGH`), operações *AND* têm prioridade média (`MEDIUM`) e operações *OR* têm a prioridade mais baixa (`LOW`). O enumerado `Priority` define as prioridades através de três constantes: `HIGH`; `MEDIUM`; e `LOW`. Uma expressão está associada ou a uma variável ou a uma operação e respetivos operandos. Um operando é outra expressão. A interface `Exp` define as operações que qualquer expressão deve implementar, nomeadamente, o método `calculate` produz o resultado da expressão, o método `toString` retorna a representação da expressão na forma de uma *string* e o método `getPriority` retorna a prioridade da expressão. Chama-se a atenção que a representação da expressão numa *string* pode envolver a utilização de parêntesis, de forma a respeitar a ordem de execução dos termos da expressão. No diagrama de classes, a característica *{final}* aplicada a uma classe significa que não se pode derivar da classe, aplicado a um campo da classe, significa que o campo não pode ser alterado depois de iniciado e aplicado a um método, significa que o método não pode ser sobreposto. A característica *{package}* significa que a classe tem visibilidade apenas dentro do *package*.



1. Considerando o diagrama estático de classes:

- Defina a interface `Exp` que reúne as operações que qualquer expressão tem de suportar.
- Defina o enumerado `Priority` com as constantes `HIGH`, `MEDIUM` e `LOW` com os valores 3, 2, e 1, respetivamente. O método `getValue` retorna o valor associado à constante.
- Defina a classe `Var`, caracterizada por um nome (`name`) e por um valor (`value`) e cumprindo os requisitos:
  - O construtor recebe os valores iniciais de `name` e `value`;
  - Os restantes métodos são métodos *accessor* e *mutator*.
- Defina a classe `BoolOp` que representa a classe base das operações booleanas. Reúne o símbolo da operação booleana (`symbol`) na forma de um caractere, e a prioridade (`priority`). Estas características são estabelecidas no construtor.
- Defina a classe `NotOp`. O construtor desta classe recebe como parâmetro a expressão a negar e o símbolo desta operação é o caractere `'!'`.
- Defina a classe `BinaryOp` que representa a classe base das operações booleanas binárias. Deve cumprir os seguintes requisitos:

- O construtor recebe como parâmetros o símbolo e a prioridade da operação, e as expressões correspondentes aos termos da esquerda e da direita do operador;
- Define dois métodos *accessor*, um para cada expressão;
- Define o método `toString` que retorna uma *string* com a representação da expressão. Chama-se a atenção que este método não pode ser sobreposto em classes derivadas.

- Defina as classes `AndOp` e `OrOp`, de acordo com os seguintes requisitos:
  - O construtor recebe como parâmetros as duas expressões correspondentes aos termos da operação;
  - Os caracteres **'&'** e **'|'** são os símbolos que representam as operações *AND* e *OR*, respetivamente.
- Considere os exemplos seguintes como representação de expressões. De realçar que a utilização de parêntesis apenas é usada para garantir a ordem das operações, tendo em consideração a prioridades dos operadores:

```
A
!A
A | B & C
(A | B) & C
!(A | !B)
A & B & C
A | B | C
```

2. Adicione aos testes unitários para este grupo um teste unitário com os seguintes requisitos:

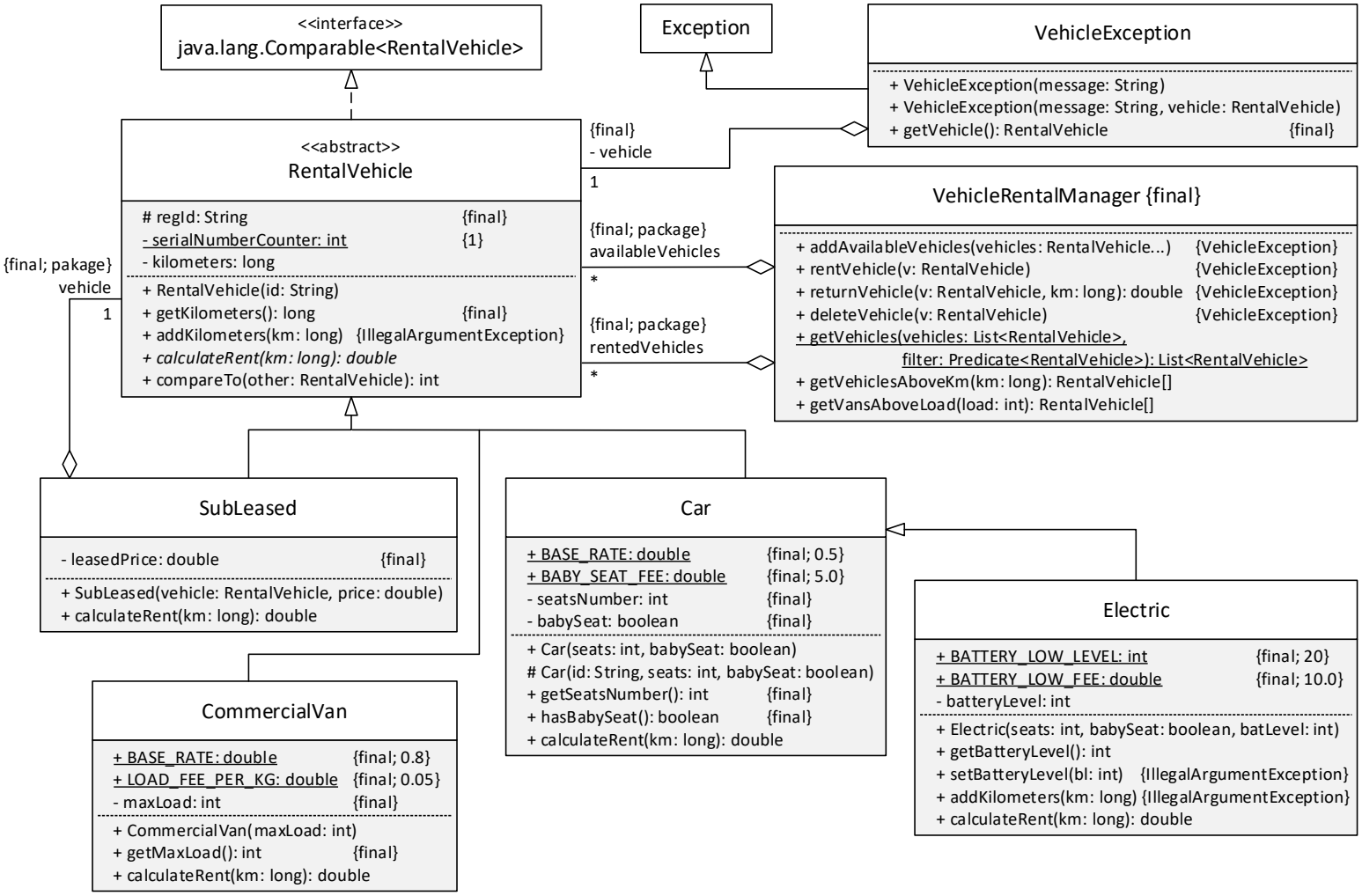
- Implemente a expressão `!((A | B & C) & (!A & B | !C))`;
- Valide o retorno da representação da expressão na forma de *string*;
- Valide o resultado da expressão para todas as combinações entre `A`, `B` e `C`.

  Nota: sugere-se a construção de um *array* de *records* Java. Considere o *record* `UnitTest` que representa uma das combinações a testar (`A`, `B` e `C`) e o resultado esperado (`R`):

```
record UnitTest(boolean A, boolean B, boolean C, boolean R) {}
UnitTest[] unitTests = {
    new UnitTest(false, false, false, <substituir por true ou false>),
    new UnitTest(false, false, true, <substituir por true ou false>),
    ...
};
```

Grupo 3

Considere o excerto de um sistema de gestão de aluguer de veículos, representado pelo seguinte diagrama de classes.



A classe `RentalVehicle` representa a classe base de todos os veículos alugáveis. Os veículos alugáveis estão organizados em diferentes tipos, nomeadamente, carrinhas de transporte de mercadorias (`CommercialVan`), veículos ligeiros (`Car`), veículos elétricos (`Electric`) que são uma especialização de veículos ligeiros e, por último, veículos subalugados a outras empresas de aluguer (`SubLeased`). A classe `VehicleRentalManager` representa a entidade que realiza a gestão dos veículos. No diagrama de classes, a característica *{final}* significa que a respetiva classe ou membro da classe é final, as características *{IllegalArgumentException, VehicleException}* significam que os respetivos métodos podem lançar exceções dos tipos indicados e as características com valores definem os valores atribuídos nos respetivos campos. Por exemplo, a constante `BATTERY_LOW_FEE`, da classe `Electric`, deve ser iniciada com o valor 10,0.

Tendo em conta o diagrama de classes:

1. Implemente a classe `RentalVehicle`, de acordo com os seguintes requisitos:

- O campo `regId` corresponde a um registo que resulta da concatenação do identificador recebido no construtor (`id`) com o valor atual do número de série (`serialNumberCounter`). O número de série é iniciado com o valor 1 e incrementado sempre que for instanciado um `RentalVehicle`. O campo `kilometers` é iniciado a 0;
- O método `getKilometers` retorna o valor atual do campo `kilometers`;
- O método `addKilometers` adiciona a `kilometers` o valor recebido no parâmetro `kilometers`. No caso do parâmetro corresponder a um valor negativo, produz a exceção `IllegalArgumentException` com a mensagem *"Kilometers cannot be negative"*;
- O método `calculateRent` é abstrato; o contrato desta operação especifica que o retorno corresponde ao custo do aluguer e depende do tipo de veículo alugado;
- Dois objetos do tipo `RentalVehicle` são comparáveis de acordo com o número de quilómetros percorridos: retorna um inteiro positivo, 0, ou um inteiro negativo se o número de quilómetros do veículo atual for superior, igual, ou inferior ao número de quilómetros do outro veículo.

2. Implemente a classe `CommercialVan`, de acordo com os seguintes requisitos:
   - O campo `maxLoad` representa a capacidade máxima de carga em quilogramas, é iniciado com o valor especificado como parâmetro do construtor e o seu valor é retornado no método `getMaxLoad`;
   - O identificador usado para construir o registo é definido pelo literal *"VAN"*;
   - O valor do aluguer é determinado pela expressão: `BASE_RATE * km + LOAD_FEE_PER_KG * maxLoad`.
3. Implemente a classe `Car`, de acordo com os seguintes requisitos:
   - Um objeto do tipo Car é caracterizado pelo número de lugares e se inclui ou não acento para bebé. Estes valores são estabelecidos através dos parâmetros dos construtores e devolvidos, respetivamente, pelos métodos `getSeatsNumber` e `hasBabySeat`;
   - Disponibiliza dois construtores: o construtor com três parâmetros recebe o identificador que estabelece o registo do veículo. Ao identificador recebido como parâmetro concatena o literal *"_CAR"*. O construtor com dois parâmetros define como identificador o literal *"CAR"*;
   - O valor do aluguer é determinado pela expressão: `BASE_RATE * km + BABY_SEAT_FEE` (se presente).
4. Implemente a classe `Electric`, de acordo com os seguintes requisitos:
   - O campo `batteryLevel` determina o valor atual de carga da bateria, expresso em percentagem (entre 0% e 100%). Este valor é estabelecido na construção através do parâmetro `batLevel`, devolvido pelo método `getBatteryLevel` e atualizado pelos métodos `setBatteryLevel` e `addKilometers`;
   - Na construção é estabelecido como identificador o literal *"ELECTRIC"*;
   - O método `setBatteryLevel` produz a exceção `IllegalArgumentException` com a mensagem *"Battery level must be between 0 and 100"* se for estabelecido como nível de carga de bateria um valor inválido (menor que 0% ou superior a 100%);
   - O método `addKilometers`, para além de adicionar o número de quilómetros estabelecidos como parâmetro, descarrega a bateria numa relação de 1% por cada 10Km percorridos, não permitindo que o valor de carga da bateria fique inferior a 0%;
   - O valor do aluguer é o mesmo que o de um veículo normal, penalizado por `BATTERY_LOW_FEE` se, no momento da entrega do veículo, o valor de carga da bateria for inferior a `BATTERY_LOW_LEVEL`.
5. Implemente a classe `SubLeased`, de acordo com os seguintes requisitos:
   - Um objeto deste tipo é caracterizado pelo veículo subalugado (`vehicle`) e pelo valor do subaluguer (`leasedPrice`), ambos estabelecidos através de parâmetros do construtor;
   - Na construção é estabelecido como identificador o literal *"SUBLEASED"*;
   - O custo do aluguer final corresponde ao custo do aluguer do veículo, e que depende do tipo concreto do veículo, acrescentado do valor do subaluguer (`leasedPrice`).
6. Implemente a classe `VehicleException`, de acordo com os seguintes requisitos:
   - Disponibiliza dois construtores: em ambos os construtores, o parâmetro `message` corresponde à mensagem devolvida pelo método `getMessage`, herdado de `Exception`. Este método não deve ser sobreposto nesta classe. O parâmetro `vehicle` corresponde ao veículo que provocou a exceção e é devolvido pelo método `getVehicle`.
7. Implemente a classe `VehicleRentalManager`, de acordo com os seguintes requisitos:
   - Agrega duas listas de veículos: (1) a lista de veículos disponíveis para alugar (`availableVehicles`); (2) a lista de veículos que estão, de momento, alugados (`rentedVehicles`). Ambas devem ser concretizadas com objetos do tipo `ArrayList`. Para efeitos de simplificação da implementação das unidades de teste, este dois campos foram definidos com visibilidade *package*.
   - O método `addAvailableVehicles` adiciona ao sistema de gestão o conjunto de veículos recebidos como parâmetro na lista `availableVehicles`. No caso de algum veículo já estar inserido no sistema, portanto, presente em qualquer uma das duas listas, é produzida a exceção `VehicleException` com a mensagem *"Vehicle is already in system"* e especificado o veículo que provocou a exceção;
   - O método `rentVehicle` aluga o veículo especificado no parâmetro do método, transitando-o da lista `availableVehicles` para a lista `rentedVehicles`. Produz a exceção `VehicleException` com a mensagem *"Vehicle is not available for rent"* se o veículo não estiver disponível para alugar e associa-o à exceção produzida;

- O método `returnVehicle` recebe como parâmetros o veículo que estava alugado e os quilómetros percorridos, transfere o veículo da lista `rentedVehicles` para a lista `availableVehicles`, atualiza o número de quilómetros no veículo e retorna o custo do aluguer. O método produz a exceção `VehicleException` com a mensagem *"Vehicle was not rented"* se o veículo não estiver presente na lista `rentedVehicles`;
- O método `deleteVehicle` elimina do sistema o veículo recebido no parâmetro, independentemente de estar na lista `availableVehicles` ou `rentedVehicles`. Produz a exceção `VehicleException` com a mensagem *"Vehicle not found in the system"* se o veículo não existir em nenhuma das duas listas;
- O método `getVehicles` retorna uma nova lista com todos os veículos da lista `vehicles` que obedecem o predicado `filter`;
- O método `getVehiclesAboveKm` retorna um *array* com todos os veículos das listas `availableVehicles` e `rentedVehicles` cujos quilómetros estejam acima do valor especificado no parâmetro `km`. O *array* deve ter os veículos ordenados de acordo com a sua ordem natural. Na implementação deste método deve definir um predicado e chamar o método `getVehicles` para cada uma das duas listas;
- O método `getVansAboveLoad` retorna um *array* com todos os veículos comerciais da lista `availableVehicles` cuja capacidade de carga seja superior ao valor do parâmetro `load`. O *array* deve ter os veículos ordenados por ordem crescente de capacidade de carga. Na implementação deste método deve recorrer novamente ao método `getVehicles` e, para ordenar o *array*, deve definir um comparador.

8. Implemente o teste unitário `getVehiclesAboveKm`, cuja declaração e requisitos de implementação estão descritos no método com o mesmo nome, no ficheiro com testes unitários *VehicleRentalManagerTest.java*.

Bom trabalho!